

SLIDE 1 — What is Array Initialization?

Definition:

Array initialization means **giving values to an array when it is created**.

Simple idea

Declare → Give Values → Store

Example:

```
int marks[5] = {80, 90, 75, 85, 95};
```

Explanation:

The values are stored in order:

```
Index: 0  1  2  3  4  
       [80] [90] [75] [85] [95]
```

 **Remember: Initialization = Giving initial values**

SLIDE 2 — Different Ways to Initialize

Method 1: Give size and values

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Method 2: Let C find the size

```
int numbers[] = {10, 20, 30, 40, 50};
```

C automatically counts **5 elements**.

Method 3: Initialize with fewer values

```
int numbers[5] = {10, 20};
```

Remaining elements become **0**.

[10] [20] [0] [0] [0]

★ Remember

- Values are stored **from left to right**
 - Unassigned elements become **0**
-

SLIDE 3 — Character Array Initialization

Arrays can also store characters.

Example

```
char vowels[5] = {'a', 'e', 'i', 'o', 'u'};  
Index:  0   1   2   3   4  
        ['a']['e']['i']['o']['u']
```

String Example

```
char name[] = "Vicky";
```

C stores the characters automatically.

V i c k y \0

📖 **Glossary:** `\0` → Null character marking the end of a C string.

🧠 **Remember:** Character array → Multiple characters

SLIDE 4 — Initializing Arrays in Programs

🎓 Student Marks

```
int marks[3] = {85, 90, 95};
```

Stores marks of 3 students.

🌡️ **Temperature**

```
float temp[3] = {30.5, 31.2, 29.8};
```

Stores temperatures.



Product Prices

```
float price[] = {99.5, 150.0, 75.5};
```

Stores multiple prices.



Basic Pattern

DECLARE → INITIALIZE → STORE → USE

SLIDE 5 — RECAP

Initialize → Give Values!

Flow

DECLARE ARRAY



GIVE VALUES



STORE VALUES



USE ARRAY



5 Things to Remember

- 1. Initialization gives initial values to an array**
- 2. Values are stored in order**
- 3. Index starts from 0**
- 4. Size can be specified or automatically determined**
- 5. Unassigned elements are initialized to 0 when applicable**



Final Example

```
int numbers[] = {10, 20, 30, 40};
```

Number of elements = 4

Output:

10 20 30 40

 **Memory Trick**

Initialize → FILL IT! 

Index → FIND IT! 

Array → STORE MANY! 

QUESTIONS

1. What is the first value of the array?

```
int numbers[3] = {10, 20, 30};
```

 **Hint:** The first element is at index 0.

 **Glossary:** Element → A value stored in an array.

 **Answer:** 10

2. Initialize a String

Complete the code to store the name "Vicky" in a character array.

```
char name__ = "Vicky";
```

```
printf("%s", name);
```

 **Hint:** Use double quotes to initialize a string.

Answer: []

3. What is the size of this array?

`int numbers[] = {10, 20, 30, 40};`

 **Hint:** Count the values.

 **Glossary:** Size → Number of elements an array contains.

 **Answer:** 4

4. What are the remaining values?

`int numbers[5] = {10, 20};`

 **Hint:** The array has 5 positions, but only 2 values are given.

[10] [20] [0] [0] [0]

 **Answer:** 0, 0, 0

5. Arrange the steps in the correct order:

- A. Store values
- B. Declare an array
- C. Use the array
- D. Give initial values

 **Hint:** First create the array, then initialize it.

 **Glossary:** Store → Keep a value in memory.

 **Answer:** B → D → A → C